JOBSHEET X

Mata Kuliah: Praktikum Pemrograman Berbasis Objek

Polimorfisme dan Exception Handling

Dosen Pengampu:

Dian Hanifudin Subhi, S.Kom., M.Kom.



Disusun oleh:

Ahmad Rafid Riqkullah

254107020078

T1 – 2B

D-IV Teknik Informatika

Tanggal: 30 Oktober 2026

POLITEKNIK NEGERI MALANG

Jl. Soekarno Hatta No.9, Jatimulyo, Kec. Lowokwaru, Kota Malang, Jawa Timur 65141

TAHUN 2026-2027

C. Langkah Kerja

Langkah 1: withdraw() Melempar InsufficientBalanceException

Code:

```java
package id.ac.polinema;

public class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException(String message) {
        super(message);
    }
}
```

Gambar 1 Kode kelas InsufficientBalanceException.java

```java
public void withdraw(double amount) throws InsufficientBalanceException {
    if (!canWithdraw(amount)) {
        throw new InsufficientBalanceException(
                accountNumber + ": insufficient balance for a withdrawal of " + amount);
    }
    balance -= amount;
}
```

Gambar 2 mplementasi method withdraw() yang melempar exception pada Account.java

```java
package id.ac.polinema;

public class Main {
    Run | Debug
    public static void main(String[] args) {
        Customer customer = new Customer("Rian", "0812-0000-0003");
        SavingsAccount savings = new SavingsAccount("A003", customer, 100000, 0.02);

        try {
            savings.withdraw(70000);
        } catch (InsufficientBalanceException e) {
            System.out.println("Withdrawal failed: " + e.getMessage());
        }

        try {
            savings.withdraw(30000);
            System.out.println("Withdrawal succeeded, new balance: " + savings.getBalance());
        } catch (InsufficientBalanceException e) {
            System.out.println("Withdrawal failed: " + e.getMessage());
        }
    }
}
```

Gambar 3 Pengujian method withdraw() dengan blok try-catch pada Main.java

Hasil:

```
PS C:\Users\IdeaPad\Documents\SEMESTER 3\Praktikum Program Berbasis Objek\Pra
ad\.antigravity-ide\extensions\redhat.java-1.58.0-win32-x64\jre\25.0.4.1-wini
sages' '-cp' 'C:\Users\IdeaPad\AppData\Roaming\Antigravity IDE\User\workspace
s\Praktikum-Pemrograman-Berbasi-Object-PBO-_2f5b4ee8\bin' 'id.ac.polinema.Mai
Withdrawal failed: A003: insufficient balance for a withdrawal of 70000.0
Withdrawal succeeded, new balance: 70000.0
PS C:\Users\IdeaPad\Documents\SEMESTER 3\Praktikum Program Berbasis Objek\Pra
```

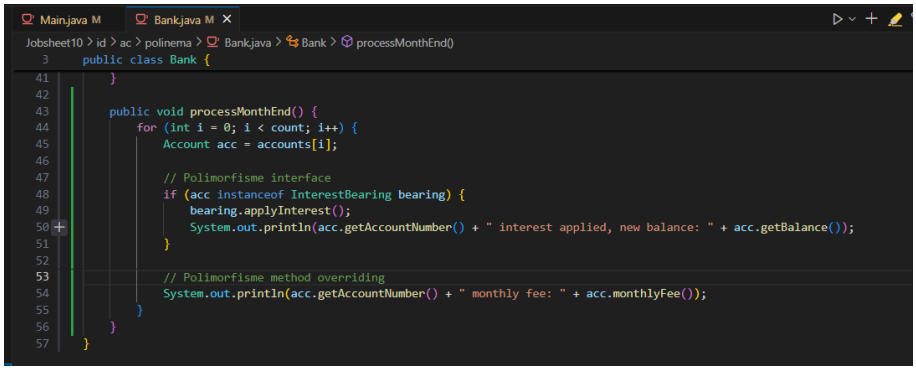
Gambar 4 Output terminal pengujian penarikan saldo (Langkah 1)

Penjelasan: Method withdraw() kini melempar InsufficientBalanceException saat penarikan tidak memenuhi syarat saldo minimum. Pada penarikan pertama sebesar 70.000, transaksi gagal karena sisa saldo akan berada di bawah batas minimum

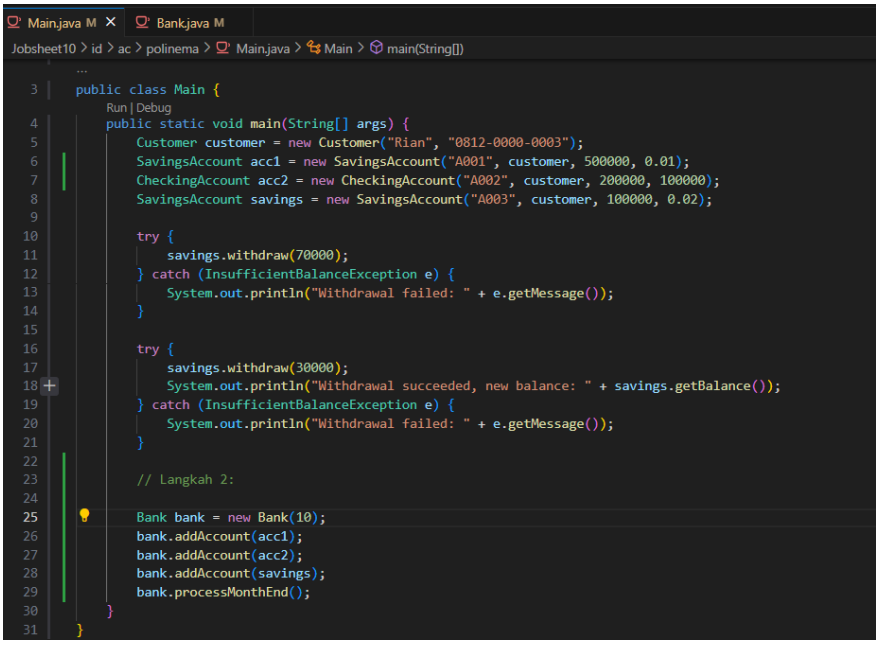
50.000. Program langsung melempar exception sebelum saldo dipotong, sehingga saldo tetap utuh (100.000) dan pesan error ditangani di blok catch. Pada penarikan kedua sebesar 30.000, transaksi berhasil karena sisa saldo mencukupi, sehingga saldo berkurang menjadi 70.000.

Langkah 2: processMonthEnd(), Polimorfisme dan instanceof

Code:

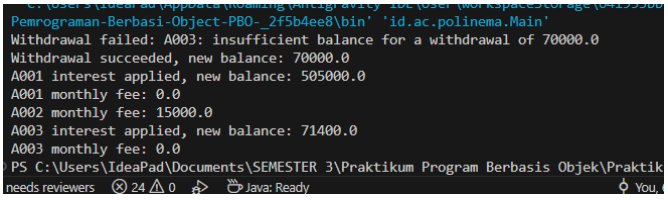


Gambar 5 Implementasi method processMonthEnd() pada Bank.java



Gambar 6 Penambahan eksekusi processMonthEnd() pada Main.java

Hasil:

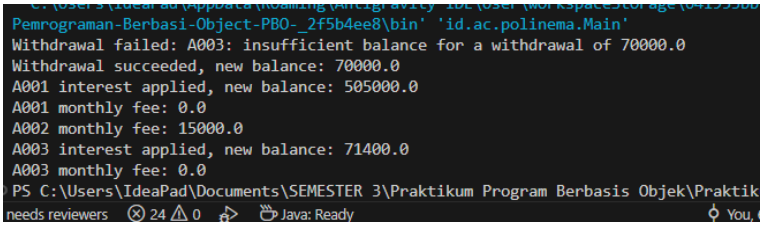


Gambar 7 Output terminal pemrosesan akhir bulan (Langkah 2)

Penjelasan: Method processMonthEnd() memanfaatkan konsep polimorfisme untuk memproses seluruh rekening dalam satu perulangan. Pemanggilan acc.monthlyFee() secara otomatis mengeksekusi method sesuai tipe asli objeknya (0.0 untuk tabungan dan 15.000 untuk giro) tanpa memerlukan percabangan if-else. Selain itu, pengecekan instanceof InterestBearing memastikan penambahan bunga hanya dijalankan pada rekening tabungan (A001 dan A003), sedangkan rekening giro dilewati.

D. Tugas dan Hasil Kerja

- Screenshot output program setelah Langkah 2.



Gambar 8 Output terminal verifikasi checkpoint Langkah 2

- Tugas mandiri:

1. Bank memerlukan laporan audit untuk rekening yang Auditable (interface dari topik Kelas Abstrak dan Interface). Tambahkan Bank.printAuditLog(): satu looping atas semua rekening, yang mencetak auditLog() hanya untuk rekening yang Auditable. Diagram berikut hanya sketsa method yang perlu ditambahkan, BUKAN kode jadi, isinya diserahkan sepenuhnya padamu:
   Code:

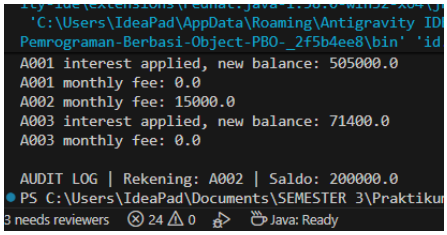
```
58      public void printAuditLog() {
59          for (int i = 0; i < count; i++) {
60              Account acc = accounts[i];
61              if (acc instanceof Auditable auditable) {
62                  System.out.println(auditable.auditLog());
63              }
64          }
65      }
66  }
```

Gambar 9 Implementasi method printAuditLog() pada Bank.java

```
// Tugas Mandiri 1
System.out.println();
bank.printAuditLog();
```

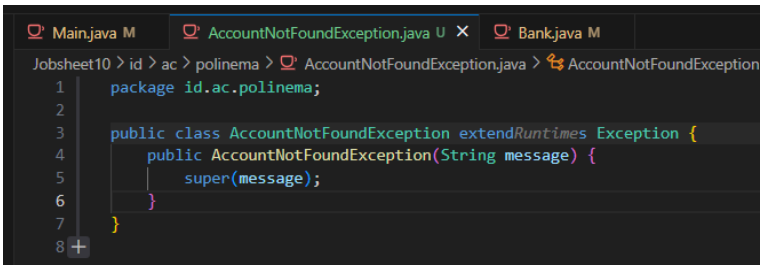
Gambar 10 Pemanggilan method printAuditLog() pada Main.java

   Hasil:

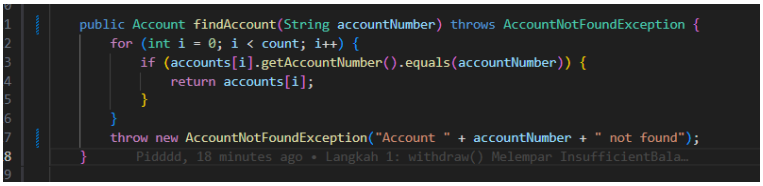


Gambar 11 Output terminal pencetakan audit log rekening giro (Tugas Mandiri 1)

2. Bank.findAccount() mengembalikan null ketika rekening tidak ditemukan. Pemanggil bisa lupa memeriksa null. Ubah supaya method itu melempar exception AccountNotFoundException. Diagram berikut hanya sketsa struktur dan signature yang berubah, BUKAN kode jadi:
   Code:



Gambar 12 Kode kelas AccountNotFoundException.java



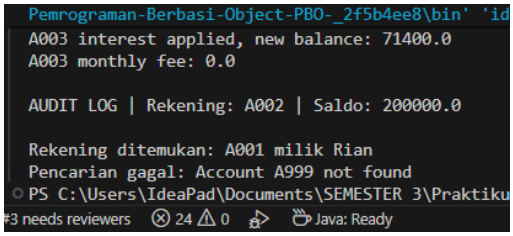
Gambar 13 Implementasi method findAccount() yang melempar exception pada Bank.java



Gambar 14 Pengujian pencarian rekening dengan blok try-catch pada Main.java

Buktikan lewat Main.java: panggil findAccount() untuk satu nomor rekening yang ada dan satu yang tidak ada, masing-masing di dalam try/catch. Hasilmu benar apabila nomor yang tidak ada menghasilkan pesan dari blok catch, dan program tetap berjalan sampai selesai.

Hasil Akhir:



Gambar 15 Output terminal pengujian (Hasil Akhir Tugas Mandiri 2)

Penjelasan: Pada Tugas Mandiri 1, printAuditLog() menggunakan instanceof Auditable sehingga hanya rekening giro (A002) yang log auditnya dicetak ke layar. Pada Tugas Mandiri 2, findAccount() melempar AccountNotFoundException jika data rekening tidak ada. Rekening A001

berhasil ditemukan dan dicetak, sedangkan rekening A999 memicu exception yang langsung ditangkap blok catch tanpa menyebabkan program *crash*.

3. Jawab secara singkat (1-2 kalimat untuk masing-masing pertanyaan):

   (a) Mengapa melempar exception lebih aman daripada mengembalikan null pada findAccount()?

   Jawab: Melempar *checked exception* mewajibkan pemanggil menangani kondisi kegagalan secara eksplisit di tingkat kompilasi (*compile-time*), sehingga mencegah potensi *runtime crash* akibat NullPointerException jika pemrogram lupa mengecek nilai kembalian null.

   (b) Mengapa processMonthEnd() memeriksa instanceof InterestBearing, bukan instanceof SavingsAccount?

   Jawab: Pengecekan berbasis interface InterestBearing memisahkan kontrak kemampuan dari hierarki kelas konkret (*loose coupling*), sehingga jika di masa depan ditambahkan jenis rekening berbunga baru (misalnya deposito atau investasi), logika method processMonthEnd() tetap dapat memprosesnya tanpa perlu diubah.

Hasil dan Pembahasan

Praktikum ini menggabungkan konsep *Exception Handling* dan *Polimorfisme*:

1. **Exception Handling:** Mengganti nilai kembalian boolean atau null dengan *custom checked exception* membuat kode lebih aman. Compiler memaksa penggunaan blok try-catch sehingga potensi *crash* akibat NullPointerException dapat dicegah sejak tahap kompilasi.
2. **Polimorfisme & instanceof:** Melalui satu tipe referensi umum (Account), sistem dapat mengeksekusi perilaku spesifik dari masing-masing subclass secara dinamis. Pengecekan kapabilitas lewat interface (InterestBearing dan Auditable) menjaga kode tetap fleksibel jika nantinya ada penambahan jenis rekening baru.

Kesimpulan

1. Penggunaan *custom checked exception* (throw, throws, try-catch) mencegah kegagalan transaksi terlewat diam-diam dan melindungi keutuhan data saldo.
2. Polimorfisme yang dipadukan dengan operator instanceof menyederhanakan pemrosesan banyak variasi objek, menghilangkan logika if-else bertingkat yang tidak perlu, dan membuat arsitektur kode lebih modular.

Lampiran

Link Github Ahmad Rafid Riqkullah atau https://github.com/Pidddd/Praktikum-Pemrograman-Berbasi-Object-PBO-/tree/main/Jobsheet10/id/ac/polinema